What are Wrapper Classes in Java?

Wrapper Classes are used to convert **primitive data types into objects**.
Why needed?

- Java collections (like ArrayList, Vector) work with **objects only**
- So we "wrap" primitive values into objects

Primitive vs Wrapper Class

Primitive Type	Wrapper Class
int	Integer
float	Float
double	Double
char	Character
boolean	Boolean
byte	Byte
short	Short
long	Long

Example – 1

```
public class WrapperExample {
   public static void main(String[] args) {
      int a = 10;              // primitive
      Integer obj = Integer.valueOf(a);  // wrapper object

      System.out.println(obj);
   }
}
```

Boxing and Unboxing

1. Boxing (Primitive → Object)

```
int a = 20;
Integer obj = Integer.valueOf(a);
```

2. Unboxing (Object → Primitive)

```
Integer obj = 30;
int a = obj.intValue();
```

3. Autoboxing

```
Integer obj = 50;   // automatic conversion
```

4. Auto-unboxing

```
Integer obj = 100;
int a = obj;   // automatic
```

Important Methods of Wrapper Classes

1. parseInt()

```
String s = "123";
```

```
        int num = Integer.parseInt(s);
```

2. valueOf()

```
        Integer obj = Integer.valueOf("456");
```

3. toString()

```
        Integer obj = 789;
        String s = obj.toString();
```

Program 1: Convert String to Integer

```
public class StringToInt {
   public static void main(String[] args) {
      String s = "100";

      int num = Integer.parseInt(s);

      System.out.println("Integer value: " + num);
   }
}
```

Program 2: Add Two Numbers Using Wrapper

```
public class AddNumbers {
   public static void main(String[] args) {
      Integer a = 10;
      Integer b = 20;

      int sum = a + b;   // auto-unboxing

      System.out.println("Sum: " + sum);
   }
}
```

Program 3: Convert Integer to String

```
public class IntToString {
   public static void main(String[] args) {
      Integer num = 500;

      String s = num.toString();

      System.out.println("String: " + s);
   }
}
```

Command Line Arguments in Java

Definition

Command Line Arguments are values passed to a Java program **at the time of execution**.
These values are received in the main() method as:
public static void main(String[] args)

Key Concept

- args is an **array of Strings**
- Each argument is stored as:
  - args[0] → first argument
  - args[1] → second argument

Example

Program

```
public class Demo {
   public static void main(String[] args) {
      System.out.println(args[0]);
      System.out.println(args[1]);
   }
}
```

Execution

```
java Demo Hello Java
```

✓ Output

```
Hello
Java
```

How It Works

Step	Explanation
Compile	javac Demo.java
Run	java Demo Hello Java
Storage	args = ["Hello", "Java"]

Important Points

- Arguments are always **String type**
- Need conversion for numbers:

```
int x = Integer.parseInt(args[0]);
```

- Index starts from **0**
- No limit on number of arguments

Programs

1. Print All Arguments

```
public class PrintArgs {
   public static void main(String[] args) {
      for(int i = 0; i < args.length; i++) {
         System.out.println(args[i]);
      }
   }
}
```

2. Add Two Numbers

```
public class Add {
   public static void main(String[] args) {
      int a = Integer.parseInt(args[0]);
      int b = Integer.parseInt(args[1]);
```

```
        System.out.println("Sum = " + (a + b));
    }
}
```

3. Count Number of Arguments

```
public class CountArgs {
    public static void main(String[] args) {
        System.out.println("Total arguments: " + args.length);
    }
}
```